

Semester	B.E. Semester VII – Computer Engineering
Subject	Scalable ML & BDA Lab
Subject In-charge	Prof. Shweta Jaiswar

Student Name	Pratik Pujari
Roll Number	23102B0008
Grade and Subject Teacher's Signature	

Experiment Number	03	
Experiment Title	Feature Engineering & Distributed ML using PySpark MLlib	
Resources / Apparatus Required	Hardware:	Software:
Description	Develop a complete machine learning workflow using PySpark MLlib. Load a structured dataset, perform feature engineering by converting categorical attributes into numerical values using StringIndexer and OneHotEncoder, combine multiple features using VectorAssembler, normalize the data using StandardScaler, split the dataset into training and testing sets, train a Logistic Regression model, generate predictions, and evaluate the model using accuracy. This experiment demonstrates how feature engineering and distributed machine learning are integrated using PySpark MLlib.	
Observation	- The Adult Census Income dataset contains both numerical and categorical attributes, making it suitable for demonstrating feature engineering techniques. - Missing categorical values represented by '?' were identified and handled during data cleaning. - Column names containing periods (.) were standardized by replacing them with underscores (_) to avoid conflicts with PySpark's nested field notation. StringIndexer successfully converted categorical variables into indexed numerical values. OneHotEncoder transformed indexed categorical features into sparse binary vectors suitable for machine learning algorithms. VectorAssembler combined numerical and encoded categorical features into a single feature vector required by Spark ML algorithms. StandardScaler normalized the feature vectors, ensuring that features with larger numeric ranges did not dominate the learning process.	

	• A Spark Pipeline successfully automated the complete preprocessing and model training workflow. • The Logistic Regression model produced reliable predictions with good classification accuracy on the test dataset, demonstrating the effectiveness of the preprocessing pipeline.
Program	<pre> # !pip install pyspark -q import kagglehub import re from pyspark.sql import SparkSession from pyspark.sql.functions import col, trim, when from pyspark.ml import Pipeline from pyspark.ml.feature import (StringIndexer, OneHotEncoder, VectorAssembler, StandardScaler) from pyspark.ml.classification import LogisticRegression from pyspark.ml.evaluation import MulticlassClassificationEvaluator # ----- # Download Dataset # ----- dataset_path = kagglehub.dataset_download("uciml/adult-census-income") # ----- # Create Spark Session # ----- spark = (SparkSession.builder .appName("Adult Census Income Classification") .master("local[*]") .getOrCreate()) # ----- # Load Dataset # ----- df = spark.read.csv(f"{dataset_path}/adult.csv", header=True, </pre>

```
inferSchema=True
)

# -----
# Rename Columns
# -----

clean_columns = [
    re.sub(r"^[a-zA-Z0-9_]", "_", c.lower())
    for c in df.columns
]

df = df.toDF(*clean_columns)

# -----
# Data Cleaning
# -----

string_columns = [
    c for c, t in df.dtypes
    if t == "string"
]

for column in string_columns:
    df = df.withColumn(
        column,
        when(trim(col(column)) == "?", None)
        .otherwise(trim(col(column)))
    )

df = df.dropna()

# -----
# Feature Columns
# -----

categorical_cols = [
    "workclass",
    "education",
    "marital_status",
    "occupation",
    "relationship",
    "race",
    "sex",
    "native_country"
]

numerical_cols = [
```

```
"age",
"fnlwgt",
"education_num",
"capital_gain",
"capital_loss",
"hours_per_week"
]

# -----
# Pipeline Stages
# -----

label_indexer = StringIndexer(
    inputCol="income",
    outputCol="label"
)

feature_indexers = [
    StringIndexer(
        inputCol=c,
        outputCol=f"{c}_index",
        handleInvalid="keep"
    )
    for c in categorical_cols
]

encoder = OneHotEncoder(
    inputCols=[f"{c}_index" for c in categorical_cols],
    outputCols=[f"{c}_vec" for c in categorical_cols]
)

assembler = VectorAssembler(
    inputCols=numerical_cols + [f"{c}_vec" for c in categorical_cols],
    outputCol="features"
)

scaler = StandardScaler(
    inputCol="features",
    outputCol="scaled_features",
    withMean=True,
    withStd=True
)

lr = LogisticRegression(
    featuresCol="scaled_features",
    labelCol="label",
    maxIter=100,
    regParam=0.01
```

```
)

pipeline = Pipeline(
    stages=[
        label_indexer,
        *feature_indexers,
        encoder,
        assembler,
        scaler,
        lr
    ]
)

# -----
# Train-Test Split
# -----

train_df, test_df = df.randomSplit([0.8, 0.2], seed=42)

# -----
# Train Pipeline
# -----

pipeline_model = pipeline.fit(train_df)

# -----
# Prediction
# -----

predictions = pipeline_model.transform(test_df)

# -----
# Evaluation
# -----

accuracy_evaluator = MulticlassClassificationEvaluator(
    labelCol="label",
    predictionCol="prediction",
    metricName="accuracy"
)

accuracy = accuracy_evaluator.evaluate(predictions)

precision = MulticlassClassificationEvaluator(
    labelCol="label",
    predictionCol="prediction",
    metricName="weightedPrecision"
).evaluate(predictions)
```

	<pre> recall = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricName="weightedRecall").evaluate(predictions) f1 = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricName="f1").evaluate(predictions) print("=" * 50) print("Model Evaluation") print("=" * 50) print(f"Accuracy : {accuracy:.4f}") print(f"Precision: {precision:.4f}") print(f"Recall : {recall:.4f}") print(f"F1 Score : {f1:.4f}") print("\nSample Predictions") predictions.select("label", "prediction", "probability").show(10, truncate=False) </pre>
Output	<pre> root -- age: integer (nullable = true) -- workclass: string (nullable = true) -- fnlwt: integer (nullable = true) -- education: string (nullable = true) -- education.num: integer (nullable = true) -- marital.status: string (nullable = true) -- occupation: string (nullable = true) -- relationship: string (nullable = true) -- race: string (nullable = true) -- sex: string (nullable = true) -- capital.gain: integer (nullable = true) -- capital.loss: integer (nullable = true) -- hours.per.week: integer (nullable = true) -- native.country: string (nullable = true) -- income: string (nullable = true) </pre> Schema of the CSV before doing any changes to data

```
-RECORD 0-----
age           | 0
workclass     | 0
fnlwgt        | 0
education     | 0
education_num | 0
marital_status | 0
occupation    | 0
relationship  | 0
race          | 0
sex           | 0
capital_gain  | 0
capital_loss  | 0
hours_per_week | 0
native_country | 0
income        | 0
```

```
workclass      : 1836
education      : 0
marital_status : 0
occupation     : 1843
relationship    : 0
race           : 0
sex            : 0
native_country : 583
income         : 0
```

But cols are present with ? string

Here the dataset marked Null values with “?”

and not NULL or UNKNOWN strings

0 Null Values with Null keywords

age	workclass	fnlwgt	education	education_num	marital_status	occupation	relationship	race	sex	capital_gain	capital_loss	hours_per_week	native_country	income
49	Self-em...	158685	HS-grad	9	Married...	Adm-cle...	Wife	White	Female	0	2377	40	United-...	>50K
34	Private	265807	HS-grad	9	Married...	transpo...	Husband	White	Male	0	2051	55	United-...	<=50K
35	Private	40135	Some-co...	10	Married...	Handler...	Husband	White	Male	0	2042	40	United-...	<=50K
26	Private	50998	Some-co...	10	Never-m...	Adm-cle...	Not-in...	White	Female	0	1974	40	United-...	<=50K
26	Private	215384	Bachelors	13	Never-m...	Exec-ma...	Not-in...	White	Male	0	1974	55	United-...	<=50K

Entire Dataset table shown above

```
Rows before cleaning : 32561
Rows after cleaning  : 30162
Rows removed         : 2399
```

Rows after data cleaning
stage

Duplicate Records : 23

Dropping duplicate rows

```
Final Dataset Shape
Rows : 30139
Columns : 15
root
|-- age: integer (nullable = true)
|-- workclass: string (nullable = true)
|-- fnlwgt: integer (nullable = true)
|-- education: string (nullable = true)
|-- education_num: integer (nullable = true)
|-- marital_status: string (nullable = true)
|-- occupation: string (nullable = true)
|-- relationship: string (nullable = true)
|-- race: string (nullable = true)
|-- sex: string (nullable = true)
|-- capital_gain: integer (nullable = true)
|-- capital_loss: integer (nullable = true)
|-- hours_per_week: integer (nullable = true)
|-- native_country: string (nullable = true)
|-- income: string (nullable = true)
```

After completing
data cleaning phase
data frame schema

Rows after cleaning: 30139

rows after cleaning in final df

Categorical Features:

['workclass', 'education', 'marital_status', 'occupation', 'relationship', 'race', 'sex', 'native_country']

Numerical Features:

['age', 'fnlwgt', 'education_num', 'capital_gain', 'capital_loss', 'hours_per_week']

Categorical and Numerical Columns in Data Set

workclass	workclass_index
State-gov	3.0
Private	0.0
Self-emp-not-inc	1.0
Local-gov	2.0
Federal-gov	5.0
Self-emp-inc	4.0
Without-pay	6.0

String Indexing sample for
a single column
“workclass”

```
root
-- age: integer (nullable = true)
-- workclass: string (nullable = true)
-- fnlwgt: integer (nullable = true)
-- education: string (nullable = true)
-- education_num: integer (nullable = true)
-- marital_status: string (nullable = true)
-- occupation: string (nullable = true)
-- relationship: string (nullable = true)
-- race: string (nullable = true)
-- sex: string (nullable = true)
-- capital_gain: integer (nullable = true)
-- capital_loss: integer (nullable = true)
-- hours_per_week: integer (nullable = true)
-- native_country: string (nullable = true)
-- income: string (nullable = true)
-- label: double (nullable = false)
-- workclass_index: double (nullable = false)
-- education_index: double (nullable = false)
-- marital_status_index: double (nullable = false)
-- occupation_index: double (nullable = false)
-- relationship_index: double (nullable = false)
-- race_index: double (nullable = false)
-- sex_index: double (nullable = false)
-- native_country_index: double (nullable = false)
-- workclass_vec: vector (nullable = true)
-- education_vec: vector (nullable = true)
-- marital_status_vec: vector (nullable = true)
-- occupation_vec: vector (nullable = true)
-- relationship_vec: vector (nullable = true)
-- race_vec: vector (nullable = true)
-- sex_vec: vector (nullable = true)
-- native_country_vec: vector (nullable = true)
```

Now applying OHE
on the string
indexed columns

And the **Schema**
after applying both
steps

```
+-----+
|features|label|
+-----+
|(104,[0,1,2,4,5,10,13,29,39,54,56,62,63],[49.0,158685.0,9.0,2377.0,40.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0])|1.0|
|(104,[0,1,2,4,5,6,13,29,43,50,56,61,63],[34.0,265807.0,9.0,2051.0,55.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0])|0.0|
|(104,[0,1,2,4,5,6,14,29,44,50,56,61,63],[35.0,40135.0,10.0,2042.0,40.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0])|0.0|
|(104,[0,1,2,4,5,6,14,30,39,51,56,62,63],[26.0,58098.0,10.0,1974.0,40.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0])|0.0|
|(104,[0,1,2,4,5,6,15,30,38,51,56,61,63],[26.0,215384.0,13.0,1974.0,55.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0])|0.0|
+-----+
only showing top 5 rows
```

Vector Assembly output

```
+-----+
|features|scaled_features|
+-----+
|(104,[0,1,2,4,5,1...|[0.80404671911021...|
|(104,[0,1,2,4,5,6...|[-0.3382511572031...|
|(104,[0,1,2,4,5,6...|[-0.2620979654489...|
|(104,[0,1,2,4,5,6...|[-0.9474766912369...|
|(104,[0,1,2,4,5,6...|[-0.9474766912369...|
+-----+
```

Standard Scaling
those vectors

```
+-----+
|scaled_features|label|
+-----+
|[0.80404671911021...|1.0|
|[-0.3382511572031...|0.0|
|[-0.2620979654489...|0.0|
|[-0.9474766912369...|0.0|
|[-0.9474766912369...|0.0|
+-----+
```

Final Data fame after
scaling

```
root
|-- features: vector (nullable = true)
|-- label: double (nullable = false)
```

Final DF before training

```
Training Records : 24014
Testing Records : 6125
```

Record Splitting

features	label	prediction	probability
[-1.4043958417623...	0.0	0.0	[0.98166099895228...
[-1.2520894582538...	0.0	0.0	[0.96615370024753...
[-1.2520894582538...	0.0	0.0	[0.99094613719778...
[-1.1759362664996...	0.0	0.0	[0.98402389485035...
[-1.0236298829912...	0.0	0.0	[0.98848094600002...
[-0.9474766912369...	0.0	0.0	[0.97121657923423...
[-0.8713234994827...	0.0	0.0	[0.98066978509340...
[-0.7190171159743...	0.0	0.0	[0.98253945477144...
[-0.6428639242200...	0.0	0.0	[0.95501389690725...
[-0.6428639242200...	0.0	0.0	[0.78679092421486...

Predictions for the Data Set

```
accuracy      : 0.8526
weightedPrecision : 0.8469
weightedRecall  : 0.8526
f1             : 0.8463
```

Accuracy and F1 Score

```
Intercept:
-1.8530202287950917

Number of Coefficients:
104
```

Coefficients of logistic model

label	prediction	count
0.0	0.0	4272
0.0	1.0	283
1.0	0.0	620
1.0	1.0	950

Some output metrics on the model

Conclusion:

A complete distributed machine learning workflow was successfully implemented using **PySpark MLlib**. The experiment demonstrated how feature engineering techniques such as **StringIndexer**, **OneHotEncoder**, **VectorAssembler**, and **StandardScaler** can be integrated into a unified **Pipeline** for efficient preprocessing and model training. Logistic Regression was trained on the processed dataset and evaluated using classification accuracy, producing satisfactory predictive performance. The experiment highlights the advantages of PySpark in building scalable machine learning

	applications capable of processing large datasets efficiently through distributed computing.
--	--